Security Review

Cantina Managed

VENUS PROTOCOL: VENUS PRS

Cantina Managed review by:
Valerian Callens, Lead Security Researcher
Chinmay Farkya, Security Researcher

CANTINA

October 27, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3
- 2.2 Codebase context . . . 3
- 2.3 Trust assumptions . . . 4
- 2.4 Cantina Managed Team Statement . . . 5

3 Findings **6**
- 3.1 Low Risk . . . 6
  - 3.1.1 Missing overflow guards in `HubCapacityMath` library . . . 6
  - 3.1.2 `HubLens` reports FRV capacity that the Hub cannot fully route . . . 6
  - 3.1.3 `HubCapacityMath` skips a feasible breakpoint and under-reports executable deposit capacity . . . 8
  - 3.1.4 `deposit()` and `mint()` settle resource interest with `_accrueFees()` before the cap check, but the views cannot . . . 8
  - 3.1.5 Using max shares amount for `withdrawPosition()` does not work for wallet-only positions . . . 9
- 3.2 Informational . . . 9
  - 3.2.1 Behavior of `maxDeposit()` and `depositCapacityBreakdown()` not aligned in `HubLens` when Hub is paused . . . 9
  - 3.2.2 Ambiguous usage of `uint256.max` sentinel in `_tvlReaching()` can cause incorrect capacity calculation . . . 10
  - 3.2.3 Flux capacity advertises Fluid sub-minimum deposit room . . . 11
  - 3.2.4 `HubCapacityMath` can report zero while a rounded percentage cascade has material executable room . . . 12
  - 3.2.5 Potential but unlikely overflow scenario introduced by PR-714 in `_calculateSeizeTokens()` for extreme inputs . . . 13
  - 3.2.6 NatSpec recommendations . . . 13
  - 3.2.7 Accidentally calling `renounceOwnership()` can lead to loss of admin ownership on the `CollateralGateway` contract . . . 14
  - 3.2.8 Deposit and supply flows in `CollateralGateway` will fail for ERC20 tokens reverting on `approve(0)` . . . 14
  - 3.2.9 Migrating a vToken position might require twice the amount of underlying assets in the related Venus Core Market . . . 15
  - 3.2.10 Flashloan fees mechanism for position migration can be improved . . . 15

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Venus Protocol is a leading lending and borrowing platform on the BNB Chain that prioritizes enhanced risk management, decentralization, and user experience.

From Oct 1st to Oct 12th the Cantina team conducted a review of venus-liquidity-hub, venus-periphery and venus-protocol on commit hashes fc2f30d9, fc2f30d9 and fc2f30d9 respectively. The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	5	5	0
Gas Optimizations	0	0	0
Informational	10	3	7
Total	**15**	**8**	**7**

2.1 Scope

Cantina reviewed the updates performed by four PRs on three distinct repositories:

- venus-liquidity-hub at PR 20.
- venus-periphery at PR 74.
- venus-protocol at PR 710.
- venus-protocol at PR 714.

The updates of the following files were in scope:

```
Repository `venus-liquidity-hub`:
├── contracts/adapters/AdapterCentrifuge.sol
├── contracts/adapters/AdapterCoreV1.sol
├── contracts/adapters/AdapterFlux.sol
├── contracts/adapters/AdapterFRV.sol
├── contracts/Hub/Hub.sol
├── contracts/interfaces/IHubLens.sol
├── contracts/interfaces/IResourceAdapter.sol
├── contracts/interfaces/IYieldGroupBase.sol
├── contracts/lens/HubCapacityMath.sol
├── contracts/lens/HubLens.sol
├── deploy/10_DeployHubLens.s.sol
├── deploy/11_DeployAdapters.s.sol
├── deploy/12_DeployHubImpl.s.sol
└── Readme.md

Repository `venus-periphery`:
├── contracts/CollateralGateway/CollateralGateway.sol
├── contracts/CollateralGateway/ICollateralGateway.sol
├── contracts/CollateralGateway/IHub.sol
├── contracts/CollateralGateway/Readme.md
├── contracts/Interfaces/IComptroller.sol
├── contracts/Interfaces/IILComptroller.sol
├── contracts/Interfaces/IPoolRegistry.sol
├── contracts/Interfaces/IVToken.sol
└── deploy/028-deploy-collateral-gateway.ts
```

```
Repository `venus-protocol`:
├── contracts/Comptroller/Diamond/facets/IMarketFacet.sol
├── contracts/Comptroller/Diamond/facets/MarketFacet.sol
├── contracts/Lens/ComptrollerLens.sol
├── contracts/Tokens/Prime/PrimeLens.sol
└── contracts/Tokens/Prime/PrimeV2.sol
```

2.2 Codebase context

The project extends the Venus Protocol to support Hub-based liquidity products and their integration with Venus lending markets. It spans three repositories:

- `venus-liquidity-hub` implements the Hub, YieldGroups, resource adapters, fixed-rate-vault integration, deposit/withdraw routing, and the stateless `HubLens` used to report deposit capacity and blended yield.
- `venus-protocol` contains the core Venus lending protocol, including the Diamond Comptroller, Prime scoring, vToken markets, liquidation logic, and protocol lenses.
- `venus-periphery` contains auxiliary integration contracts. The relevant addition is `CollateralGateway`, which composes Hub deposits, Venus market minting, collateral entry, and collateral migration into user-facing flows.

The reviewed changes are organized into four pull requests.

PR 20 - venus-liquidity-hub: This PR moves Hub deposit-capacity calculation from the Hub's hot path into the stateless `HubLens` and the pure `HubCapacityMath` library. The solver evaluates each YieldGroup's absolute and percentage caps against the post-deposit TVL, allowing percentage-capped Hubs to bootstrap while preserving the Hub's routing and capacity checks. `HubLens` also exposes queue-aligned capacity breakdowns and a TVL-weighted spot APY.

The PR additionally updates the FRV adapter to report supplier APY net of the reserve factor, adds the `resourceName` view to resource adapters, and supplies the deployment changes required for the new adapters, Hub implementation, and lens.

PR 710 - venus-protocol: This PR adds the ACM-gated `MarketFacet.enterMarketForAccount` entry point needed by integrations such as `CollateralGateway` to enter a Venus market for a user without a separate user delegation transaction.

It also updates Prime V2 and Prime Lens to support underlying assets with up to 24 decimals by scaling values correctly instead of assuming decimals do not exceed 18. The storage layout is unchanged.

PR 74 - venus-periphery: This PR adds `CollateralGateway`, a stateless integration contract that can deposit assets into a Hub, mint the corresponding Venus receipt-market tokens, and enter the market for the user in one transaction. It also supports flash-loan-assisted collateral migration for positions backing a borrow and supply-and-enter flows for Isolated Pool markets.

The Core-market path depends on the `enterMarketForAccount` functionality introduced by PR 710. The gateway uses transient storage for in-flight migration state and has no persistent protocol state; its owner can only sweep stray tokens.

PR 714 - venus-protocol: This PR fixes precision loss in `ComptrollerLens._calculateSeizeTokens`. The previous implementation truncated an intermediate price ratio to 18 decimals before multiplying by the repayment amount, which could produce zero seized collateral for liquidations involving high-decimal debt assets. The revised calculation multiplies first and divides once, preserving precision while retaining downward rounding for the final vToken amount.

The change applies to all Comptroller Lens seize-calculation entry points and requires redeploying and selecting the updated plain lens contract. It is particularly relevant before receipt-market positions introduced through the Hub can be liquidated safely.

2.3 Trust assumptions

The review was conducted under the following trust assumptions:

- Governance and operational actors are assumed to perform their duties in accordance with documented procedures, mandates, and responsibilities. These actors are assumed to perform appropriate due diligence before authorizing on-chain actions.
- End users are considered untrusted and may behave arbitrarily or maliciously within the permissions granted by the system. Users are responsible for reviewing and understanding the on-chain actions they submit.
- The review assumes adherence to the principle of Segregation of Duties. No single actor is expected to simultaneously hold operational roles intended to remain independent, and collusion between incompatible roles is considered out of scope unless explicitly stated otherwise.
- The Liquidity Hub is assumed to support only standard ERC-20 tokens. Support for non-standard ERC-20 tokens (e.g., rebasing tokens, fee-on-transfer tokens, or tokens implementing transfer hooks) is outside the scope of this review.
- Partial compliance of the Hub with ERC-4626 standard is intended, and `HubLens` is intended to cover these gaps. In particular:
  - for the value returned by `maxDeposit()`, it is intended that it must not be higher than the actual maximum that would be accepted (it should underestimate if necessary), and it must factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily), it must return 0.
  - for the value returned by `maxMint()`, it is intended that it must return the maximum amount of shares mint would allow to be deposited to receiver and not cause a revert, which must not be higher than the actual maximum that would be accepted (it should underestimate if necessary), and that it must factor in both global and user-specific limits, like if mints are entirely disabled (even temporarily), it must return 0.
- The user needs to delegate the gateway contract within the comptroller logic, for the position migration to work (mintBehalf and redeemBehalf require delegation in the Comptroller). The delegate grant is pool wide: it lets this gateway redeem and borrow against any Core position the caller holds, not only the vh market. But the grant outlives the call and the caller needs to revoke it with `updateDelegate(gateway, false)`.
- Position migration via the `CollateralGateway` can fail due to paused markets, supply caps in hub routing, redeem/mint/flashloan actions paused on the relevant markets.

2.4 Cantina Managed Team Statement

Venus engaged Cantina to conduct a security review of four PRs (PR 710, PR 74, PR 714, PR 20) on three different repositories (venus-protocol, venus-periphery, venus-liquidity-hub). We would like to thank the Venus team for their responsiveness and constructive engagement throughout the review process. No significant issues were identified during the assessment, and the system is expected to operate as intended.

3 Findings

3.1 Low Risk

3.1.1 Missing overflow guards in `HubCapacityMath` library

Severity: Low Risk.

Context: HubCapacityMath.sol#L8-L11.

Description: The NatSpec of `HubCapacityMath.sol` states that *a test can drive it with cap books a live Hub would refuse to accept (* `absoluteCap == type(uint256).max` *,* `percentageCapBps > 10_000` *) and still reach the saturation and overflow guards.*

However, two main aspects can result in overflows in that library:

1. The `_satAdd()` function could be used for the following arithmetic operations where an overflow can happen:

```
// in _toPoints()
tracked += g.balance;

// in_segmentAt()
base += p.absBound;
base += p.balance;

// in _roomAt()
if (held > balance) room += held - balance;
```

2. The `_mulBpsDown()` function has a path that can overflow in theory. Still, it can only happen for invalid Hub configurations where `percentageCapBps > 10_000`:

```
/// @dev `x × bps / 10_000`, rounded down, without overflowing on a large `x`.
function _mulBpsDown(uint256 x, uint256 bps) private pure returns (uint256 scaled) {
    if (bps == 0) return 0;
    if (x <= type(uint256).max / bps) return (x * bps) / BPS_DENOMINATOR;
    scaled = (x / BPS_DENOMINATOR) * bps + ((x % BPS_DENOMINATOR) * bps) /
    ↪  BPS_DENOMINATOR;
}
```

With `bps == type(uint16).max == 65,535` and an extreme value for `x = type(uint256).max / 2`:

- The fast path is skipped because `x * bps` overflows:

```
if (x <= type(uint256).max / bps) return (x * bps) / BPS_DENOMINATOR;
```

- The fallback then evaluates `(x / 10_000) * 65_535` which also overflows and reverts:

```
scaled = (x / BPS_DENOMINATOR) * bps + ((x % BPS_DENOMINATOR) * bps) /
↪  BPS_DENOMINATOR;
```

Recommendation: Consider:

1. Using `_satAdd()` instead of `+=` in the four items listed above.
2. Making `_mulBpsDown()` overflow-safe for all `uint16` BPS inputs, including values above `10_000`, or documenting the fact that the overflow scenario is known and accepted, but that values above `10_000` are currently invalid for Hub configurations.

Venus Protocol: Fixed in PR 37.

Cantina Managed: Both recommended items were fixed in PR 37 through commits 66baa83 and 5ec8a32.

3.1.2 `HubLens` reports FRV capacity that the Hub cannot fully route

Severity: Low Risk.

Context: YieldGroupBase.sol#L437-L456.

Description: `IYieldGroupBase.maxDeposit()` intentionally reports venue room before the Hub cap. The interface also allows `YieldGroupBase.deposit()` to place less than requested, and FRV intentionally permits a sub-minimum amount when it exactly fills a residual tail. Those are documented behaviors. The problem is that `HubLens.maxDeposit()` applies the Hub cap and presents the resulting amount as a Hub-wide maximum, even though the ordered mutation may skip part of it and cause `HubCapacityExceeded`.

Two manifestations are covered:

1. Outer-cap sub-floor: the FRV group has room above its minimum, but the Hub percentage/absolute cap clips the amount sent to it below the minimum. The Lens reports the clipped amount. The FRV cascade skips it.
2. Tail-before-next-resource: the first FRV resource has an exact sub-minimum tail. The mutating cascade consumes that tail, leaving a sub-minimum remainder for the next resource, which is skipped. Omitting the tail from the venue-only group view is intentional. The Hub Lens fails to account for that omitted tail when claiming that the aggregate Hub amount is fully routable.

Code snippets: `IYieldGroupBase.maxDeposit()` is intentionally venue-only and does not include the Hub cap:

```
/// Must satisfy: deposit(maxDeposit()) does not revert.
/// Venue room only — NOT the Hub's cap on this YieldGroup.
function maxDeposit() external view returns (uint256 capacity);
```

FRV's implementation omits a sub-minimum resource room from the aggregate view, because a caller cannot generally send that amount as a standalone leg:

```
uint256 room = _resourceDepositRoom(resource, cfg.adapter);
uint256 minDep = _minDeposit(resource);
total += room >= minDep ? room : 0;
```

The mutation intentionally allows a sub-minimum amount when it exactly fills the resource's remaining room, and otherwise skips it:

```
uint256 room = _resourceDepositRoom(resource, adapter);
uint256 toDeposit = remaining < room ? remaining : room;

if (toDeposit < room && toDeposit < _minDeposit(resource)) return 0;
```

The Lens passes the venue-only aggregate room into `HubCapacityMath` and applies the Hub cap without modeling whether the resulting outer amount is fully executable by the ordered inner queue:

```
venueRoom = IYieldGroupBase(yieldGroup).maxDeposit();

inputBuffer[count] = HubCapacityMath.GroupInput({
    balance: balance,
    venueRoom: venueRoom,
    absoluteCap: cfg.absoluteCap,
    percentageCapBps: cfg.percentageCapBps
});
```

Proof of Concept: The POC in `tests/foundry/fork/Fork_POC_HubLensFRVMinimum.t.sol` contains two fork tests. Both create fresh Fundraising FRV vaults and route the FRV YieldGroup as the only outer deposit destination. To be run, `.env` should be created by copying `.env.example`.

- Create `tests/foundry/fork/Fork_POC_HubLensFRVMinimum.t.sol` and add the following:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.25;

// solhint-disable func-name-mixedcase, ordering, import-path-check

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { IERC20Metadata } from
↪  "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";

import { IHub } from "../../../contracts/interfaces/IHub.sol";
import { IYieldGroupBase } from "../../../contracts/interfaces/IYieldGroupBase.sol";
import { IFRVVault } from "../../../contracts/interfaces/external/IFRVVault.sol";

import {
    FRVFixture,
    IAccessControlled,
    IACM,
    IOwnable,
    IFRVController,
    IFRVLifecycle,
    InstitutionalConfig,
    RiskConfig
} from "./base/FRVFixture.sol";

/**
 * @title POC: HubLens reports FRV capacity that the Hub cannot fully route
 * @notice The two tests cover the following two manifestations:
 *         - an outer percentage cap creates a non-tail amount below the FRV
 ↪  minimum;
 *         - an exact tail is consumed before a later resource, leaving a
 ↪  sub-minimum remainder.
 *
 * The FRV minimum, partial-fill behavior, and exact-tail exception are intentional
 ↪  YieldGroup
 * semantics. The finding is that HubLens presents the resulting outer amount as a
 ↪  fully routable
 * Hub maximum, although the ordered Hub/YieldGroup cascade cannot place it in full.
 */
contract Fork_POC_HubLensFRVMinimum is FRVFixture {
    uint256 internal constant FRV_MINIMUM = 100;
    uint256 internal constant OUTER_IDLE = 50;
    uint16 internal constant HALF_BPS = 5_000;

    function _wantsFullTopology() internal pure override returns (bool) {
        return true;
    }

    function _seedScenario() internal override { }

    function test_percentageCapCreatesUnexecutableSubFloor() public onlyFork
    ↪  onlyFresh {
        Sys storage s = usdt;
        address vault = _createFundraisingVault(s, FRV_MINIMUM, 1_000_000,
        ↪  "percentage");
        _wireFrvResource(s, vault);

        // The absolute cap is deliberately non-binding. With 50 idle units and a
        ↪  50% cap,
        // X = 50 + floor(X / 2) gives a post-deposit TVL of 100 and capacity of 50.
```

```
        vm.prank(governance);
        s.hub.lowerYieldGroupCap(address(s.ygFrv), HUGE_CAP, HALF_BPS);
        _fund(s.asset, address(s.hub), _amt(s, OUTER_IDLE));
        _setOuterQueues(s, _addrs(address(s.ygFrv)));

        uint256 reported = lens.maxDeposit(address(s.hub));
        assertEq(reported, _amt(s, OUTER_IDLE), "Lens reports the sub-minimum outer
        ↪  capacity");
        assertGt(IYieldGroupBase(address(s.ygFrv)).maxDeposit(), _amt(s,
        ↪  FRV_MINIMUM));

        _fund(s.asset, alice, reported);
        vm.startPrank(alice);
        s.asset.approve(address(s.hub), reported);
        vm.expectRevert(abi.encodeWithSelector(IHub.HubCapacityExceeded.selector,
        ↪  reported, 0));
        s.hub.deposit(reported, alice);
        vm.stopPrank();

        assertEq(IYieldGroupBase(address(s.ygFrv)).totalAssets(), 0, "FRV received
        ↪  nothing");
        assertEq(s.asset.balanceOf(alice), reported, "failed deposit is atomic");
    }

    function test_exactTailIsConsumed_thenNextSubFloorIsSkipped() public onlyFork
    ↪  onlyFresh {
        Sys storage s = usdt;
        address vaultA = _createFundraisingVault(s, 50, 100, "tailA");
        address vaultB = _createFundraisingVault(s, 50, 60, "tailB");

        vm.startPrank(governance);
        s.ygFrv.addResource(vaultA, address(frvAdapter));
        s.ygFrv.addResource(vaultB, address(frvAdapter));
        s.ygFrv.setInnerDepositQueue(_addrs(vaultA, vaultB));
        s.ygFrv.setInnerWithdrawQueue(_addrs(vaultA, vaultB));
        if (!_hubHasGroup(s.hub, address(s.ygFrv))) {
            s.hub.addYieldGroup(address(s.ygFrv), HUGE_CAP, PCT_DISABLED);
        }
        vm.stopPrank();

        _setOuterQueues(s, _addrs(address(s.ygFrv)));

        // Fill A to 70, leaving an exact 30-unit tail. B retains 60 units of room.
        _seedDeposit(s, alice, _amt(s, 70));
        assertEq(IFRVVault(vaultA).maxDeposit(address(s.ygFrv)), _amt(s, 30));
        assertEq(IFRVVault(vaultB).maxDeposit(address(s.ygFrv)), _amt(s, 60));

        uint256 reported = lens.maxDeposit(address(s.hub));
        assertEq(reported, _amt(s, 60), "Lens reports B's venue room");

        // The mutation consumes A's exact 30-unit tail. The remaining 30 is below
        ↪  B's minimum,
        // so B is skipped and the Hub reverts after placing 30.
        _fund(s.asset, bob, reported);
        vm.startPrank(bob);
        s.asset.approve(address(s.hub), reported);
        vm.expectRevert(
            abi.encodeWithSelector(IHub.HubCapacityExceeded.selector, reported,
            ↪  _amt(s, 30))
        );
        s.hub.deposit(reported, bob);
        vm.stopPrank();

        // The Hub transaction is atomic, so the exact tail is still present after
        ↪  the revert.
```

```
    assertEq(IYieldGroupBase(address(s.ygFrv)).totalAssets(), _amt(s, 70));
}

function _wireFrvResource(Sys storage s, address vault) internal {
    vm.startPrank(governance);
    s.ygFrv.addResource(vault, address(frvAdapter));
    s.ygFrv.setInnerDepositQueue(_addrs(vault));
    s.ygFrv.setInnerWithdrawQueue(_addrs(vault));
    if (!_hubHasGroup(s.hub, address(s.ygFrv))) {
        s.hub.addYieldGroup(address(s.ygFrv), HUGE_CAP, PCT_DISABLED);
    }
    vm.stopPrank();
}

function _createFundraisingVault(Sys storage s, uint256 minimum, uint256
↪  maxBorrowCap, string memory suffix)
    internal
    returns (address vault)
{
    require(cfg.frvController != address(0), "POC: FRV controller unavailable");
    address collateral = cfg.frvCollateral;
    require(collateral != address(0) && collateral != address(s.asset), "POC:
    ↪  bad collateral");
    address institution_ = makeAddr(string.concat("institution-", suffix));

    _prepareFrvController();
    uint256 idealCollateral = 100 * (10 **
    ↪  IERC20Metadata(collateral).decimals());

    vault = IFRVController(cfg.frvController).createVault(
        _vaultConfig(s, minimum, maxBorrowCap),
        _institutionConfig(collateral, idealCollateral, institution_),
        _riskConfig(),
        string.concat("POC FRV ", suffix),
        string.concat("poc", suffix),
        string.concat("POC Institution ", suffix)
    );

    deal(collateral, institution_, idealCollateral);
    vm.startPrank(institution_);
    IERC20(collateral).approve(vault, idealCollateral);
    IFRVLifecycle(vault).depositCollateral(idealCollateral);
    vm.stopPrank();

    IFRVController(cfg.frvController).openVault(vault);
}

function _prepareFrvController() internal {
    address frvAcm =
    ↪  IAccessControlled(cfg.frvController).accessControlManager();
    vm.mockCall(frvAcm, abi.encodeWithSelector(IACM.isAllowedToCall.selector),
    ↪  abi.encode(true));

    address positionToken = cfg.frvPositionToken;
    if (IOwnable(positionToken).owner() != cfg.frvController) {
        vm.prank(IOwnable(positionToken).owner());
        IOwnable(positionToken).transferOwnership(cfg.frvController);
        IFRVController(cfg.frvController).acceptPositionTokenOwnership();
    }
}

function _vaultConfig(Sys storage s, uint256 minimum, uint256 maxBorrowCap)
    internal
    view
    returns (IFRVVault.FRVVaultConfig memory config)
{
```

```
        config = IFRVVault.FRVVaultConfig({
            supplyAsset: IERC20(address(s.asset)),
            fixedAPY: FRV_APY_BPS,
            reserveFactor: 0,
            minBorrowCap: _amt(s, 1),
            maxBorrowCap: _amt(s, maxBorrowCap),
            minSupplierDeposit: _amt(s, minimum),
            openDuration: 7 days,
            lockDuration: 30 days,
            settlementWindow: 7 days
        });
    }

    function _institutionConfig(address collateral, uint256 idealCollateral,
    ↪  address institution_)
        internal
        pure
        returns (InstitutionalConfig memory config)
    {
        config = InstitutionalConfig({
            collateralAsset: IERC20(collateral),
            idealCollateralAmount: idealCollateral,
            marginRate: 1e18,
            institutionOperator: institution_,
            positionTokenId: 0
        });
    }

    function _riskConfig() internal pure returns (RiskConfig memory config) {
        config = RiskConfig({
            liquidationThreshold: 0.85e18,
            liquidationIncentive: 1.1e18,
            latePenaltyRate: 1.1e18
        });
    }
}
```

- Run:

```
forge test --match-contract POC_HubLensFRVMinimum -vv
```

- Result:

```
Ran 2 tests for
↪  tests/foundry/fork/Fork_POC_HubLensFRVMinimum.t.sol:Fork_POC_HubLensFRVMinimum
[PASS] test_exactTailIsConsumed_thenNextSubFloorIsSkipped() (gas: 6175955)
[PASS] test_percentageCapCreatesUnexecutableSubFloor() (gas: 3618224)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 10.44s (11.86s CPU
↪  time)

Ran 1 test suite in 10.45s (10.44s CPU time): 2 tests passed, 0 failed, 0 skipped (2
↪  total tests)
```

Recommendation: Consider keeping `IYieldGroupBase.maxDeposit()` as venue-only capacity, including its documented partial-fill and exact-tail semantics. Add a separate order-aware Hub-level feasibility calculation for `HubLens.maxDeposit()` and `depositCapacityBreakdown()`. It must simulate the outer cap together with the inner queue, minimum-deposit floor, partial fills, and exact-tail exception, and return only an amount for which the Hub can fully route the deposit. Do not change the meaning of `YieldGroupBase.maxDeposit()` to include the Hub cap.

Venus Protocol: Fixed in PR 37.

Cantina Managed: PR 37 fixes this issue by making the lens dry-run the Hub routing, including the

FRV floor and tail rule, and reports only a figure that routes in full. `YieldGroupBase.maxDeposit()` is unchanged.

3.1.3 `HubCapacityMath` skips a feasible breakpoint and under-reports executable deposit capacity

Severity: Low Risk.

Context: HubCapacityMath.sol#L258-L281.

Description: `HubCapacityMath._solveCeiling` can skip a feasible shared breakpoint when adjacent sub-100% segment fixed points straddle it, so `HubLens.maxDeposit` and `depositCapacityBreakdown` may under-report executable capacity. The live Hub can still route the deposit, so integrators may reject valid deposits or miss a materially executable route. The likelihood of that scenario is limited but it could impact integrators that may reject valid deposits or show zero capacity, due to conservative under-reporting.

Proof of Concept: The proof of concept registers the two yield groups, seeds the Hub with the idle `IDLE` balance via a plain ERC-20 transfer, then compares `lens.maxDeposit(address(hub))` and `lens.depositCapacityBreakdown(address(hub))` against the live deposit path. The witness shows `maxDeposit == 0` while `_depositAs(alice, DEPOSIT)` routes the full `3,000.536193029490616620`-token deposit in queue order. The expected outcome would be that the lens should return a capacity that the Hub can actually route.

- Create new file `tests/foundry/Integration/POC_HubCapacityBreakpoint.t.sol` and add the following:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.25;

// solhint-disable func-name-mixedcase, ordering, import-path-check

import { console } from "forge-std/console.sol";

import { IntegrationBase } from "./IntegrationBase.t.sol";

/**
 * @title POC: HubCapacityMath skips a feasible sub-100% breakpoint
 * @notice Proof statement: With a supported 18-decimal cap book, HubLens reports
 ↪  zero capacity
 *         while the production Hub routes the exact 3,000.536193029490616620-token
 ↪  deposit through
 *         the real Hub, YieldGroups, and adapters. The witness has a first-segment
 ↪  fixed point one
 *         wei above the shared breakpoint and a next-segment fixed point one wei
 ↪  below it; the
 *         the solver skips both adjacent segment candidates and does not evaluate
 ↪  the shared breakpoint neighborhood.
 *
 * The leaf venues are deterministic test doubles supplied by IntegrationBase; the
 ↪  target path
 * exercised here is the production Hub -> HubLens / HubCapacityMath -> YieldGroup
 ↪  -> adapter path.
 */
contract POC_HubCapacityBreakpoint is IntegrationBase {
    uint256 internal constant GROUP_A_ABS = 100e18;
    uint16 internal constant GROUP_A_PCT = 1;
    uint256 internal constant GROUP_B_ABS = 3000e18;
    uint16 internal constant GROUP_B_PCT = 5595;

    uint256 internal constant IDLE = 2361394101876675603218;
    uint256 internal constant BREAKPOINT = 5361930294906166219840;
    uint256 internal constant POST_TVL = BREAKPOINT - 2;
    uint256 internal constant DEPOSIT = POST_TVL - IDLE;
```

```
    uint256 internal constant GROUP_A_ROOM = 536193029490616621;
    uint256 internal constant GROUP_B_ROOM = 2999999999999999999;

    function test_lensSkipsBreakpoint_butHubRoutesDeposit() public {
        _registerCoreResources();
        _registerFluxResources();
        _registerHubYieldGroup(address(ygCore), GROUP_A_ABS, GROUP_A_PCT);
        _registerHubYieldGroup(address(ygFlux), GROUP_B_ABS, GROUP_B_PCT);

        address[] memory queue = _addrs(address(ygCore), address(ygFlux));
        _setDepositQueue(queue);
        _setWithdrawQueue(queue);

        // A plain ERC-20 transfer creates the immovable Hub idle balance; no
        ↪  privileged state is
        // needed for this prerequisite.
        _mintAsset(alice, IDLE);
        vm.prank(alice);
        asset.transfer(address(hub), IDLE);

        assertGe(ygCore.maxDeposit(), GROUP_A_ABS, "Core venue room must not bind");
        assertGe(ygFlux.maxDeposit(), GROUP_B_ABS, "Flux venue room must not bind");

        uint256 breakpoint = (GROUP_B_ABS * BPS + GROUP_B_PCT - 1) / GROUP_B_PCT;
        uint256 firstFixedPoint = (IDLE * BPS) / (BPS - GROUP_A_PCT - GROUP_B_PCT);
        uint256 nextFixedPoint = ((IDLE + GROUP_B_ABS) * BPS) / (BPS - GROUP_A_PCT);
        assertEq(breakpoint, BREAKPOINT, "witness breakpoint");
        assertEq(firstFixedPoint, BREAKPOINT + 1, "first crossing is above
        ↪  breakpoint");
        assertEq(nextFixedPoint, BREAKPOINT - 1, "next crossing is below
        ↪  breakpoint");

        uint256 reported = lens.maxDeposit(address(hub));
        (address[] memory groups, uint256[] memory room) =
        ↪  lens.depositCapacityBreakdown(address(hub));

        console.log("HubLens.maxDeposit", reported);
        console.log("HubLens.breakdown[0]", room[0]);
        console.log("HubLens.breakdown[1]", room[1]);
        console.log("Executable deposit", DEPOSIT);

        assertEq(reported, 0, "the solver skips both sides of the breakpoint");
        assertEq(groups[0], address(ygCore), "breakdown preserves Core queue
        ↪  order");
        assertEq(groups[1], address(ygFlux), "breakdown preserves Flux queue
        ↪  order");
        assertEq(room[0], 0, "zero aggregate produces no Core leg");
        assertEq(room[1], 0, "zero aggregate produces no Flux leg");

        _mintAndApprove(alice, DEPOSIT);
        _depositAs(alice, DEPOSIT);

        assertEq(hub.totalAssets(), POST_TVL, "the live Hub reaches the feasible
        ↪  post-deposit TVL");
        assertEq(ygCore.totalAssets(), GROUP_A_ROOM, "the first group receives its
        ↪  floored room");
        assertEq(ygFlux.totalAssets(), GROUP_B_ROOM, "the residual reaches the
        ↪  second group");
        assertEq(ygCore.totalAssets() + ygFlux.totalAssets(), DEPOSIT, "all deposit
        ↪  assets are routed");
        assertEq(asset.balanceOf(address(hub)), IDLE, "the original idle TVL remains
        ↪  on the Hub");
        _assertSolvent();
    }
}
```

- Run:

```
forge test --match-contract POC_HubCapacityBreakpoint --match-test
↪  'test_lensSkipsBreakpoint_butHubRoutesDeposit' -vv
```

- Result:

```
Ran 1 test for tests/foundry/Integration/POC_HubCapacityBreakpoint.t.sol:POC_HubCap┘
↪  acityBreakpoint
[PASS] test_lensSkipsBreakpoint_butHubRoutesDeposit() (gas: 3891764)
Logs:
  HubLens.maxDeposit 0
  HubLens.breakdown[0] 0
  HubLens.breakdown[1] 0
  Executable deposit 3000536193029490616620

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 16.86ms (3.30ms CPU
↪  time)

Ran 1 test suite in 23.07ms (16.86ms CPU time): 1 tests passed, 0 failed, 0 skipped
↪  (1 total tests)
```

Recommendation: Treat every finite breakpoint as a candidate edge and evaluate the exact integer cascade at that edge before moving to the next segment. For example, after a segment crossing falls above `high`, test `high` with `_roomAt(points, high)` (or carry the greatest feasible edge forward), and preserve the maximum feasible edge across adjacent segments. Keep the existing per-group rounding check so the reported edge is executable under the separately floored caps.

Venus Protocol: Fixed in PR 29.

Cantina Managed: PR 29 fixes this issue by replacing the `if (exact < low || exact > high) continue` logic: when the crossing lands above `high`, the solver now tests the segment's top edge with `_raiseIfFits(points, floorTvl, ceiling, high - 1)` and keeps it if the cascade admits it.

3.1.4 `deposit()` and `mint()` settle resource interest with `_accrueFees()` before the cap check, but the views cannot

Severity: Low Risk.

Context: Hub.sol#L712-L721.

Description: `HubLens.maxDeposit()` calculates a deposit capacity using a stale NAV, while the actual `deposit()` will first execute `_accrueFees()`.

That gap can become problematic if the capacity of group `G` is percent-based, let's say 50% of the hub TVL:

- During the simulation, `HubLens.maxDeposit()` calculates the maximum amount to deposit, to still fit 50% of the hub TVL, let's say `x`.
- However, when trying to deposit `x`, we will call `_accrueFees()`, the NAV will slightly increase, which will increase the current proportion of `G`, with regards to the whole Hub, and then depositing `x` will exceed that percent-based capacity of `G`, so we can revert with `HubCapacityExceeded`.

Note that it also applies to `HubLens.maxMint()`.

Recommendation: Consider making the Core adapter return a conservative bound.

Venus Protocol: Fixed in PR 29.

Cantina Managed: PR 29 addresses this issue by trimming each `YieldGroup`'s room by 0.1% of its mark (twice for `maxMint`, since the settled interest also raises the share price) through commits 162d2ce and ba767b8. An accrual past that margin still reverts; this is documented in `IHubLens` and covered by tests.

3.1.5 Using max shares amount for `withdrawPosition()` does not work for wallet-only positions

Severity: Low Risk.

Context: CollateralGateway.sol#L147-L170, CollateralGateway.sol#L355-L364.

Description: As per the `CollateralGateway` README file, if input shares amount in `withdrawPosition()` is `type(uint256).max`, then it withdraws the whole position.

This is the relevant logic:

```
uint256 walletShares = IERC20(hub).balanceOf(msg.sender);
if (walletShares > shares) walletShares = shares;
if (walletShares != 0) IERC20(hub).safeTransferFrom(msg.sender, address(this),
↪  walletShares);

uint256 freedShares;
if (walletShares < shares) freedShares = _freeFromMarket(hub, vhMarket, shares -
↪  walletShares);
```

If the user has Hub shares only in their wallet and no shares in the form of vhMarket core receipts:

- `shares == type(uint256).max`, so `walletShares < shares`.
- The remaining `shares - walletShares` is still nonzero and call flow reaches `_freeFromMarket()`.

This is the code for `_freeFromMarket()`:

```
function _freeFromMarket(address hub, address vhMarket, uint256 shares) private
↪  returns (uint256 freedShares) {
    uint256 rate = IVToken(vhMarket).exchangeRateCurrent();
    uint256 balance = IVToken(vhMarket).balanceOf(msg.sender);

    uint256 vTokens = balance;
    if (shares <= (balance * rate) / EXP_SCALE) vTokens = ((shares * EXP_SCALE) +
    ↪  rate - 1) / rate;

    uint256 balanceBefore = IERC20(hub).balanceOf(address(this));
    uint256 errorCode = IVToken(vhMarket).redeemBehalf(msg.sender, vTokens);
    if (errorCode != 0) revert VTokenRedeemFailed(vhMarket, errorCode);
```

Since vhMarket receipt balance for user is `0`, `_freeFromMarket()` attempts to redeem zero Core receipts via `redeemBehalf()` and it reverts because zero redemption reverts at the vToken's end.

Thus, max withdrawal does not work for a wallet-only position.

Recommendation: Consider going through the Core redemption only if the user holds non-zero receipt balance for the given vhMarket. This will allow using `shares == type(uint256).max` as an input for withdrawals that involve wallet-only positions.

Venus Protocol: Fixed in commit d0a5976.

Cantina Managed: Fix verified. Commits 6bffa62 and d0a5976 fix this issue so that a wallet-only position with `shares = max` now redeems just the wallet shares. In the position withdrawal logic, the core leg is not run if the `vhMarket` balance of user is zero, or the user has not set the gateway as an approved delegate, which allows the wallet shares to still complete the hub withdrawal.

3.2 Informational

3.2.1 Behavior of `maxDeposit()` and `depositCapacityBreakdown()` not aligned in `HubLens` when Hub is paused

Severity: Informational.

Context: HubLens.sol#L38-L51.

Description: In `HubLens.sol`, the behavior of `maxDeposit()` and `depositCapacityBreakdown()` is not aligned when `IHub(hub).hubPaused()` returns `true`.

`maxDeposit()` checks if the Hub is paused first:

```
if (IHub(hub).hubPaused()) return 0;
```

However, `depositCapacityBreakdown()` first calls `_readDepositQueue()` before checking `IHub(hub).hubPaused()`:

```
(groups, inputs, hubTotalAssets) = _readDepositQueue(hub);

if (IHub(hub).hubPaused()) return (groups, new uint256[](groups.length));
```

`_readDepositQueue()` calls `hub.totalAssets()` and also reads queue configuration and YieldGroup views. Therefore, with a paused Hub:

1. `depositCapacityBreakdown()` enters `_readDepositQueue()`.
2. `_readDepositQueue()` calls `totalAssets()`.
3. If NAV or another required view reverts, the whole breakdown reverts.
4. The paused return path is never reached.

This differs from `maxDeposit()`, which returns zero without reading NAV or queue data.

The issue exists if paused behavior is intended to be fail-fast and return zero capacity.

Recommendation: Consider checking `hubPaused()` before `_readDepositQueue()` in `depositCapacityBreakdown()`. Returning the queue addresses with a zero-filled room array is otherwise coherent, but that behavior should be explicitly documented and tested.

Venus Protocol: Fixed in PR 37.

Cantina Managed: Fix verified. PR 37 moves the `hubPaused()` check ahead of `_readDepositQueue()` and returns empty arrays while paused, since the Hub takes no deposits then.

3.2.2 Ambiguous usage of `uint256.max` sentinel in `_tvlReaching()` can cause incorrect capacity calculation

Severity: Informational.

Context: HubCapacityMath.sol#L340-L348.

Description: `_tvlReaching()` returns `uint256.max` when the intermediate multiplication overflows:

```
/// @dev `[target × 10_000 / pctBps]`, rounded up so the breakpoint is exact against
↪  the floored
    ///      cap the cascade computes. `type(uint256).max` when out of range.
    function _tvlReaching(uint256 target, uint16 pctBps) private pure returns
    ↪  (uint256 tvl) {
        if (pctBps == 0) return type(uint256).max;
```

```
        if (target > type(uint256).max / BPS_DENOMINATOR) return type(uint256).max;
        uint256 numerator = target * BPS_DENOMINATOR;
        tvl = numerator / pctBps;
        if (numerator % pctBps != 0) ++tvl;
    }
```

Later code treats that value as a real breakpoint, like in `_holdAt()`:

```
/// @dev What one YieldGroup holds once TVL is `tvl`, i.e. `hold_i` of the header:
///      `max(balance, min(absBound, pctBps × tvl / 10_000))`. The breakpoints
↪  already say which
///      of the three cases applies, so nothing is compared here.
function _holdAt(CapPoint memory p, uint256 tvl) private pure returns (uint256
↪  held) {
    if (tvl >= p.breakHigh) return p.absBound;
    if (tvl < p.breakLow) return p.balance;
    held = _mulBpsDown(tvl, p.pctBps);
}
```

This can produce two incorrect outcomes:

- `_holdAt()` may ignore an absolute cap and make the lens advertise a deposit that causes `Hub.deposit()` to revert.
- `_solveCeiling()` may skip a final segment whose upper bound is `uint256.max`, under-reporting or returning zero capacity even though deposits remain executable.

That scenario is very unlikely to happen in practice. Still, it is reported as an existing edge case of the library.

Proof of Concept: The proof of concept covers both manifestations of the sentinel collision:

1. In the overstatement test, it sets up the valid extreme cap, seeds `M / 2` idle underlying, reads `lens.maxDeposit(address(hub))`, and calls `hub.deposit(reported, alice)`. The lens returns an inflated maximum, and the public deposit reverts on the Hub's checked `ta * 9_999` calculation.
2. In the under-reporting test, it registers two valid groups with `M / 2` absolute caps and 50% percentage caps. Both absolute breakpoints collapse to the `uint256.max` sentinel, so the solver skips the final 100%-slope segment and reports zero capacity; nevertheless, a `1e18` deposit succeeds and is routed evenly between the two groups.

The expected behavior is that `HubLens.maxDeposit()` must never advertise a deposit that the Hub rejects and must not discard executable capacity merely because a breakpoint is not representable through an intermediate multiplication.

- Create new file `tests/foundry/Integration/POC_HubCapacityMath.t.sol` and add the following:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.25;

// solhint-disable func-name-mixedcase, ordering, import-path-check

import { HubBase } from "../Hub/HubBase.t.sol";

/**
 * @title POC: HubLens extreme-cap breakpoint errors
 * @notice Proof statements:
 *         1. HubLens.maxDeposit can overstate capacity because _tvlReaching marks a
 *            representable absolute breakpoint as unreachable, while the
 ↪  corresponding
```

```
 *            public Hub.deposit call reverts during its checked percentage-cap
 ↪  calculation.
 *         2. HubLens.maxDeposit can understate capacity because _solveCeiling skips
 ↪  a final
 *            100%-slope segment whose upper edge is the same unreachable-breakpoint
 ↪  sentinel,
 *            while the corresponding public Hub.deposit call succeeds.
 *
 * Bug vector:
 * 1. Governance registers two valid caps: (M / 5_000, 9_999 BPS) and (M / 10_000, 1
 ↪  BPS).
 * 2. The Hub already contains M / 2 idle underlying, and both routed groups
 ↪  advertise their
 *    respective absolute caps as venue room.
 * 3. HubLens treats Group A's breakpoint as M because target * 10_000 overflows, so
 ↪  it reports
 *    roughly M / 2 of deposit capacity even though Group A's absolute bound is
 ↪  already binding.
 * 4. An unprivileged depositor funding and approving the advertised amount calls
 ↪  Hub.deposit and
 *    the production routing path reverts on ta * 9_999 before any value is placed.
 */
contract POC_HubLensCapacity is HubBase {
    function test_maxDeposit_understatesFinalUnboundedSegment_butDepositSucceeds()
     ↪  public {
        uint256 max = type(uint256).max;
        uint256 absoluteCap = max / 2;
        uint256 depositAmount = 1e18;
        uint256 halfDeposit = depositAmount / 2;

        // Both caps are valid Hub configurations. Because absoluteCap * 10_000
         ↪  overflows,
        // _tvlReaching(absBound, 5_000) returns uint256.max for both groups. The
         ↪  solver therefore
        // sees one final segment with slopeBps == 10_000 and high == uint256.max,
         ↪  then skips it.
        _addYieldGroup(srcA, absoluteCap, 5_000);
        _addYieldGroup(srcB, absoluteCap, 5_000);
        _setDepositQueue(_addrs(address(srcA), address(srcB)));
        srcA.setMaxDeposit(absoluteCap);
        srcB.setMaxDeposit(absoluteCap);

        assertEq(lens.maxDeposit(address(hub)), 0, "solver skips the final unbounded
         ↪  segment");

        _mintAndApprove(alice, depositAmount);
        _depositAs(alice, depositAmount);

        assertEq(hub.totalAssets(), depositAmount, "the live Hub accepts the
         ↪  under-reported deposit");
        assertEq(srcA.totalAssets(), halfDeposit, "first group receives half of the
         ↪  deposit");
        assertEq(srcB.totalAssets(), halfDeposit, "second group receives the
         ↪  remainder");
    }

    function test_maxDeposit_overstatesExtremeAbsoluteBound_andDepositReverts()
     ↪  public {
        uint256 max = type(uint256).max;
        uint256 groupAAbsolute = max / 5_000;
        uint256 groupBAbsolute = max / BPS;
        uint256 initialIdle = max / 2;

        _addYieldGroup(srcA, groupAAbsolute, 9_999);
        _addYieldGroup(srcB, groupBAbsolute, 1);
```

```
        _setDepositQueue(_addrs(address(srcA), address(srcB)));
        srcA.setMaxDeposit(groupAAbsolute);
        srcB.setMaxDeposit(groupBAbsolute);
        _mintAsset(address(hub), initialIdle);

        uint256 reported = lens.maxDeposit(address(hub));
        uint256 groupBBreakpoint = groupBAbsolute * BPS;
        uint256 expectedReported = groupBBreakpoint - initialIdle;

        assertEq(reported, expectedReported, "sentinel breakpoint overstates the
        ↪  lens result");
        assertGt(reported, groupAAbsolute + groupBAbsolute, "reported room exceeds
        ↪  absolute bounds");

        _mintAndApprove(alice, reported);
        vm.prank(alice);
        vm.expectRevert();
        hub.deposit(reported, alice);
    }
}
```

- Run:

```
forge test --match-contract POC_HubLensCapacity -vv
```

- Result:

```
Ran 2 tests for
↪  tests/foundry/Integration/POC_HubLensCapacity.t.sol:POC_HubLensCapacity
[PASS] test_maxDeposit_overstatesExtremeAbsoluteBound_andDepositReverts() (gas:
↪  993082)
[PASS] test_maxDeposit_understatesFinalUnboundedSegment_butDepositSucceeds() (gas:
↪  1140946)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.22ms (826.96µs CPU
↪  time)

Ran 1 test suite in 6.39ms (2.22ms CPU time): 2 tests passed, 0 failed, 0 skipped (2
↪  total tests)
```

Recommendation: Consider:

- Representing an "unreachable breakpoint" separately from a numeric breakpoint, for instance by using an explicit boolean.
- Implementing overflow-safe `ceil(target * BPS / pctBps)`.
- Clamping `_holdAt()` to `min(absBound, percentageRoom)`.
- Handling the final unbounded solver segment explicitly.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. The collision is real, but it needs a `YieldGroup`'s absolute bound above `type(uint256).max / 10_000` (about 1.16e73 wei, or 1.16e55 tokens at 18 decimals), and `absBound` is also capped by `balance + venueRoom`. No real configuration comes near the trigger. Neither outcome puts funds at risk: in the over-report case `Hub.deposit()` reverts atomically, and in the under-report case the lens is only conservative.

3.2.3 Flux capacity advertises Fluid sub-minimum deposit room

Severity: Informational.

Context: AdapterFlux.sol#L121-L128.

Description: `AdapterFlux.maxDeposit()` forwards Fluid's raw headroom, but the Flux `YieldGroup` does not model Fluid's `minDeposit()`. A positive residual created by the YieldGroup's resource cap can therefore be reported as usable capacity, so `HubLens.maxDeposit()` and `depositCapacityBreakdown()` may overstate the ceiling or name Flux as the route even though execution skips the resource and the Hub reverts atomically.

Proof of Concept: The POC seeds real `fUSDT`, tightens the Flux resource cap to the already-held balance plus `minDeposit() - 1`, and shows that `YieldGroup.maxDeposit()`, `HubLens.maxDeposit()`, and `depositCapacityBreakdown()` all advertise that residual, even though a real `Hub.deposit()` cannot place it and reverts atomically.

- Create the file `tests/foundry/fork/Fork_POC_FluxMinDeposit.t.sol` and add:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.25;

// solhint-disable func-name-mixedcase, ordering, import-path-check

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { IHub } from "../../../contracts/interfaces/IHub.sol";
import { IFToken } from "../../../contracts/interfaces/external/IFToken.sol";

import { ForkBase } from "./base/ForkBase.sol";

interface IFluidFTokenMinimum {
    function minDeposit() external view returns (uint256);
}

/**
 * @title POC: Flux advertises a Fluid sub-minimum residual
 * @notice Proof statement: Using the real BSC Fluid fUSDT and the real production
 ↪  Hub, YieldGroup,
 *         and AdapterFlux, prove that a positive resource-cap residual below
 ↪  fUSDT.minDeposit() is
 *         returned by HubLens.maxDeposit() but cannot be deposited. The Hub must
 ↪  revert atomically,
 *         demonstrating the capacity/routing mismatch without using mocks or
 ↪  altered production code.
 *
 * Bug vector:
 * 1. A normal Flux position is seeded, then its resource cap is tightened to leave
 ↪  minDeposit() - 1.
 * 2. AdapterFlux/YieldGroup/HubLens expose that positive residual as executable
 ↪  capacity.
 * 3. A user deposits exactly the advertised amount; Fluid rejects the sub-minimum
 ↪  deposit and the
 *    Hub reverts with no state or balance change.
 */
contract Fork_POC_FluxMinDeposit is ForkBase {
    function test_fork_flux_advertisesSubMinimumResidual() public onlyFork
    ↪  onlyFresh withFlux {
        Sys storage s = usdt;

        uint256 minimum = IFluidFTokenMinimum(s.fToken).minDeposit();
        uint256 rawRoom = IFToken(s.fToken).maxDeposit(address(this));
        emit log_named_uint("fork block", block.number);
        emit log_named_address("fUSDT", s.fToken);
        emit log_named_uint("Fluid minDeposit", minimum);
        emit log_named_uint("Fluid maxDeposit", rawRoom);
        assertGt(minimum, 1, "fUSDT minimum must leave a positive sub-minimum
        ↪  value");
```

```
        assertGt(rawRoom, minimum, "fUSDT must have protocol headroom above its
        ↪  minimum");

        _registerFluxYieldGroup(s, HUGE_CAP, PCT_DISABLED);
        _setOuterQueues(s, _addrs(address(s.ygFlux)));

        uint256 seed = _amt(s, 1000);
        _seedDeposit(s, alice, seed);

        // Tightening a resource cap to the already-held value plus one less than
        ↪  Fluid's minimum
        // models the ordinary residual left by a finite configured cap.
        uint256 held = s.ygFlux.totalAssets();
        uint256 residual = minimum - 1;
        vm.prank(governance);
        s.ygFlux.lowerResourceCap(s.fToken, held + residual);

        assertEq(s.ygFlux.maxDeposit(), residual, "YieldGroup advertises the
        ↪  sub-minimum residual");
        assertEq(lens.maxDeposit(address(s.hub)), residual, "HubLens advertises the
        ↪  sub-minimum residual");
        (address[] memory groups, uint256[] memory rooms) =
        ↪  lens.depositCapacityBreakdown(address(s.hub));
        assertEq(groups.length, 1, "Flux is the only routable group");
        assertEq(groups[0], address(s.ygFlux), "breakdown points to Flux");
        assertEq(rooms[0], residual, "breakdown advertises the sub-minimum
        ↪  residual");
        emit log_named_uint("advertised residual", residual);

        uint256 beforeAssets = s.hub.totalAssets();
        uint256 beforeUserBalance = IERC20(address(s.asset)).balanceOf(bob);
        _fund(s.asset, bob, residual);

        vm.startPrank(bob);
        s.asset.approve(address(s.hub), residual);
        vm.expectRevert(abi.encodeWithSelector(IHub.HubCapacityExceeded.selector,
        ↪  residual, 0));
        s.hub.deposit(residual, bob);
        vm.stopPrank();

        assertEq(s.hub.totalAssets(), beforeAssets, "sub-minimum deposit changed Hub
        ↪  NAV");
        assertEq(IERC20(address(s.asset)).balanceOf(bob), beforeUserBalance +
        ↪  residual, "deposit was not atomic");
        assertEq(s.ygFlux.totalAssets(), held, "sub-minimum deposit changed Flux
        ↪  holdings");
        assertEq(IERC20(s.fToken).balanceOf(address(s.ygFlux)) > 0, true, "seed
        ↪  position remains in Fluid");
    }
}
```

- Run:

```
forge test --match-contract Fork_POC_FluxMinDeposit -vv
```

- Result:

```
Ran 1 test for
↪  tests/foundry/fork/Fork_POC_FluxMinDeposit.t.sol:Fork_POC_FluxMinDeposit
[PASS] test_fork_flux_advertisesSubMinimumResidual() (gas: 2941709)
Logs:
  fork block: 127439062
  fUSDT: 0xA5b8FCa32E5252B0B58EAbf1A8c79d958F8EE6A2
```

```
  Fluid minDeposit: 33554432
  Fluid maxDeposit: 170141183460467911598786828765242446925
  advertised residual: 33554431

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.12s (1.23s CPU time)

Ran 1 test suite in 5.13s (5.12s CPU time): 1 tests passed, 0 failed, 0 skipped (1
↪  total tests)
```

Recommendation: Consider exposing Fluid's `minDeposit()` through the Fluid interface and the adapter/YieldGroup capacity logic. The effective room must be checked against that minimum after the YieldGroup's own per-resource cap is applied. For example, the adapter can reject raw Fluid room below the floor:

```
function maxDeposit(address resource) external view override returns (uint256) {
    uint256 room = IFToken(resource).maxDeposit(address(this));
    return room < IFToken(resource).minDeposit() ? 0 : room;
}
```

Because the proof of concept's residual is created by the `YieldGroup` cap, the adapter check alone is insufficient. Keep the cap calculation in asset units and add a generic minimum-feasibility hook (or an equivalent post-cap check) so both `maxDeposit()` and the mutating cascade treat a sub-minimum effective room consistently. If a separate route is desired for sub-minimum amounts, the capacity lens and queue solver must model that route explicitly rather than reporting it as Flux capacity.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. It needs the room left for Flux (resource cap or Hub cap minus what Flux holds) to fall below Fluid's `minDeposit()`, which is a dust-sized amount. No Flux resource cap is set today. If hit, the deposit reverts atomically, so there is no fund risk.

3.2.4 `HubCapacityMath` can report zero while a rounded percentage cascade has material executable room

Severity: Informational.

Context: HubCapacityMath.sol#L266-L281.

Description: `HubCapacityMath._solveCeiling` can drop a valid fixed-point neighborhood when the first exact crossing fails `_roomAt` only because each percentage cap is floored independently. With one unit of immovable Hub TVL and two groups at `5,000` / `4,997` BPS, the lens can return `0` even though the ordered cascade can execute `3,331` units. The breakdown inherits the same zero result. The likelihood of that scenario is limited but it could impact integrators that may reject valid deposits or show zero capacity, due to conservative under-reporting.

Proof of Concept: The proof of concept uses the production `HubCapacityMath` library with two queued groups at `5,000` and `4,997` BPS, one unit of immovable Hub TVL, and large absolute/venue room. It shows `HubCapacityMath.capacity` and `capacityPerGroup` returning zero while a `3,331`-unit deposit produces post-deposit TVL `3,332` and fully consumes the two floored percentage rooms. The expected behavior would be that the lens should return at least `3,331`.

- Create new file `tests/foundry/Integration/POC_HubCapacityMath.t.sol` and add the following:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.25;

import { Test } from "forge-std/Test.sol";
```

```
import { HubCapacityMath } from "../../../contracts/lens/HubCapacityMath.sol";

/**
 * @title POC: percentage-cap rounding can collapse Hub capacity to zero
 * @notice Proof statement: with one unit of immovable Hub TVL and two valid
 *         percentage-bound groups at 5,000 and 4,997 BPS, the production
 *         HubCapacityMath solver returns zero capacity even though the exact
 *         cascade can place 3,331 units. The per-group vector is also zeroed
 *         because it inherits that aggregate result.
 *
 * Bug vector:
 * 1. The first segment has base = 1, slope = 9,997 BPS, and two independent
 *    percentage floors.
 * 2. Its closed-form crossing is 3,333, but the exact cascade has only 3,331
 *    room there, so the solver enters the pre-payment fallback.
 * 3. Since base (1) <= pctBoundCount (2), the segment is discarded and later
 *    segments have crossings below their segment floors.
 * 4. The solver returns the pre-deposit TVL (1), hence capacity zero, although
 *    a 3,331-unit deposit yields post-deposit TVL 3,332 and routes fully.
 */
contract POC_HubCapacityMath_3ae8579c is Test {
    uint16 internal constant BPS = 10_000;

    function test_roundingFallbackReportsZeroButCascadeHasRoom() public pure {
        HubCapacityMath.GroupInput[] memory groups = new
        ↪  HubCapacityMath.GroupInput[](2);
        groups[0] = HubCapacityMath.GroupInput({
            balance: 0, venueRoom: type(uint256).max, absoluteCap: 1_000_000,
            ↪  percentageCapBps: 5_000
        });
        groups[1] = HubCapacityMath.GroupInput({
            balance: 0, venueRoom: type(uint256).max, absoluteCap: 1_000_000,
            ↪  percentageCapBps: 4_997
        });

        uint256 preDepositTvl = 1;
        uint256 reported = HubCapacityMath.capacity(groups, preDepositTvl);
        (uint256 total, uint256[] memory perGroup) =
        ↪  HubCapacityMath.capacityPerGroup(groups, preDepositTvl);

        assertEq(reported, 0, "solver incorrectly reports nonzero capacity");
        assertEq(total, 0, "breakdown total inherits solver result");
        assertEq(perGroup[0], 0, "first group inherits zero aggregate result");
        assertEq(perGroup[1], 0, "second group inherits zero aggregate result");

        uint256 executableDeposit = 3_331;
        uint256 postDepositTvl = preDepositTvl + executableDeposit;
        uint256 firstRoom = (postDepositTvl * 5_000) / BPS;
        uint256 secondRoom = (postDepositTvl * 4_997) / BPS;

        assertEq(postDepositTvl, 3_332, "post-deposit TVL");
        assertEq(firstRoom, 1_666, "first independent percentage floor");
        assertEq(secondRoom, 1_665, "second independent percentage floor");
        assertEq(firstRoom + secondRoom, executableDeposit, "ordered cascade
        ↪  consumes the deposit");

        uint256 nextPostDepositTvl = postDepositTvl + 1;
        uint256 nextRoom = (nextPostDepositTvl * 5_000) / BPS
            + (nextPostDepositTvl * 4_997) / BPS;
        assertLt(preDepositTvl + nextRoom, nextPostDepositTvl, "3,332-unit deposit
        ↪  is not executable");
    }
}
```

- Run:

```
forge test --match-contract POC_HubCapacityMath --match-test
↪  'test_roundingFallbackReportsZeroButCascadeHasRoom' -vv
```

- Result:

```
Ran 1 test for
↪  tests/foundry/Integration/POC_HubCapacityMath.t.sol:POC_HubCapacityMath_3ae8579c
[PASS] test_roundingFallbackReportsZeroButCascadeHasRoom() (gas: 30922)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 7.17ms (438.04µs CPU
↪  time)

Ran 1 test suite in 16.55ms (7.17ms CPU time): 1 tests passed, 0 failed, 0 skipped
↪  (1 total tests)
```

Recommendation: Preserve the nearest feasible boundary instead of abandoning the segment when `base <= pctBoundCount`. The solver should evaluate the largest integer TVL around the failed exact crossing, or use an integer fixed-point calculation that accounts for the individual floors, and then verify it with `_roomAt`. The returned value must remain within the segment and be checked against the actual room before becoming `ceiling`.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. The skip only fires when `base <= pctBoundCount` (1-2 wei), and the under-report is then below `k × 10_000` wei, in the lens only. On mainnet it is unreachable: every Hub is seeded at launch with 10 tokens to dead address, so base can never get that low.

3.2.5 Potential but unlikely overflow scenario introduced by PR-714 in `_calculateSeizeTokens()` for extreme inputs

Severity: Informational.

Context: ComptrollerLens.sol#L349.

Description: In `ComptrollerLens.sol`, PR-714 changed the calculation of `_calculateSeizeTokens()` to:

```
uint numerator = mul_(mul_(actualRepayAmount, liquidationIncentiveMantissa),
↪  priceBorrowedMantissa);
uint denominator = mul_(priceCollateralMantissa, exchangeRateMantissa);
seizeTokens = div_(numerator, denominator);
```

This fixes an intermediate precision loss, but it multiplies the repayment amount before division. The old implementation first reduced the liquidation-incentive/price ratio to an 18-decimal `Exp` and only then multiplied by the repayment amount.

Therefore, an overflow becomes possible in theory for the result of the multiplication `mul_(mul_(actualRepayAmount, liquidationIncentiveMantissa), priceBorrowedMantissa)` when using extreme inputs.

Recommendation: Consider confirming that no live scenario can hit this condition. Based on that assessment, the direct three-factor multiplication could be replaced with a checked `mulDiv` /cancellation approach that preserves the exact floor without requiring the full numerator to fit in 256 bits.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. Solidity 0.8 checked math reverts on overflow, and it needs a repay worth over ~1e23 USD, far beyond any real debt. A revert undoes the whole liquidation, so no funds are at risk.

3.2.6 NatSpec recommendations

Severity: Informational.

Context: CollateralGateway.sol#L50-L52, ICollateralGateway.sol#L61.

Description: NatSpec can be improved at several places in the codebase :

1. The description for transient migration state mentions that all migration variables are updated prior to the flashloan execution, but two variables, `migrationShares` and `migrationVtokens` are actually written after the flashloan is executed.
2. The description for input `assets` in `SuppliedToSpoke` event is `Underlying that reached the market, after any transfer fee`, but within the `_supplyAndEnterSpokeMarket` flow, the underlying is also transferred to the market contract which will take a transfer fee again, eventually decreasing the actual asset amount that reaches the market. This event description can be changed to `Underlying that reached the gateway, after any transfer fee`.

Recommendation: Consider modifying the NatSpec as suggested.

Venus Protocol: Fixed in PR 83.

Cantina Managed: Fix verified. PR 83 addresses both issues through commits ae289fb (correcting the `assets` description to reflect the amount that reached the gateway) and 32ea004 (clarifying which fields are written before the flash loan and that `_migrationShares` and `_migrationVTokens` are written during it).

3.2.7 Accidentally calling `renounceOwnership()` can lead to loss of admin ownership on the `CollateralGateway` contract

Severity: Informational.

Context: CollateralGateway.sol#L7.

Description: The `CollateralGateway` contract inherits from OpenZeppelin's `Ownable2Step` library.

We have the `renounceOwnership` function from this `Ownable2Step` library. If it is called by mistake, ownership of the contract will be lost forever.

Recommendation: Consider overriding the `renounceOwnership()` function to always revert.

Venus Protocol: Fixed in PR 83.

Cantina Managed: Fix verified. PR 83 overrides `renounceOwnership()` as a no-op, so ownership can't be renounced by mistake.

3.2.8 Deposit and supply flows in `CollateralGateway` will fail for ERC20 tokens reverting on `approve(0)`

Severity: Informational.

Context: CollateralGateway.sol#L288-L298.

Description: At many instances in the contract, `forceApprove()` is used to reset allowances to required amount. This is done to support approvals for tokens like USDT that require a zero amount approval before the actual approve call.

This does not support certain tokens like BNB on Ethereum that always revert on `approve(0)` calls.

```
    function _depositAndSupply(
        address hub,
        address asset,
        address vhMarket,
        uint256 assets,
```

```
        uint256 minShares,
        address receiver
    ) private returns (uint256 shares, uint256 vTokens) {
        IERC20(asset).forceApprove(hub, assets);
        shares = IHub(hub).deposit(assets, address(this));
==>     IERC20(asset).forceApprove(hub, 0);
```

A `forceApprove(0)` call is guaranteed to revert for such tokens.

Recommendation: Consider verifying that such tokens are not intended to be supported by Venus on any chains that it is deployed on, and document this assumption.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. The gateway currently targets BNB Chain only, and every token it approves there accepts a zero approval. A port to another chain would revisit the zero-approval resets for such tokens.

3.2.9 Migrating a vToken position might require twice the amount of underlying assets in the related Venus Core Market

Severity: Informational.

Context: CollateralGateway.sol#L254-L276.

Description: In the flashloan-related path for migrating a vToken position,

- First, the flashloan amount is taken from old vToken market, for example vUSDT Core Market;
- Then that underlying token(lets say USDT) goes to the hub;
- Then it executes redemption from old vUSDT market, to get USDT;
- Then it repays the USDT back to the same vUSDT market;
- Both the flashloan and the redemption steps check that `getCashPrior()` needs to be greater than the requested loan/redemption amount;

Since the flashloaned amount is only returned back to the vToken market after the redemption proceeds are already in the gateway contract, this means that to successfully execute the entire flashloan-based migration path ⇒ the initial `internalCash` balance needed is ~twice the amount of underlying assets constituting the user's collateral (vUSDT) in that market.

This happens because :

- Flashloan is taken from the same Venus core market, where we need to redeem the vToken collateral position from.
- This happens in the same flow before any underlying assets get back to the vToken contract.

This can lead to failure of migrations where available internalCash is < 2x of what the user's position is worth.

Note : This happens only in the cases where the hub deposit does not route back the USDT to the same Venus Core market, and the USDT instead moves to a different resource via the Hub.

Recommendation: The fund settlement logic can be improved for the migrations to succeed even when the underlying Venus core market has < ~2x of the required cash balance.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. The mechanism is confirmed, but the 2x figure does not hold in practice. The borrowed underlying is deposited into the Hub inside the same callback, and the Hub routes it back into the source market before the redeem runs. The order in a migration trace is `transferOutUnderlyingFlashLoan`, then `Hub.deposit`, which reaches `vUSDT.mint` through the Core yield source, and only then `redeemBehalf`. The market's `getCash()` is back at its starting value by the time the redeem checks it. What the market needs depends on where the Hub routes the

deposit at that moment: a non-Core source and the 2x holds, the Core source and 1x is enough. The gateway cannot know which beforehand, so a fixed pre-flight check would reject migrations that succeed today. No fund risk either way: the call reverts atomically and the caller keeps their position.

3.2.10 Flashloan fees mechanism for position migration can be improved

Severity: Informational.

Context: CollateralGateway.sol#L134-L135.

Description: In the flashloan-based migration path, a flashloan is taken for the entire underlying amount that is worth the user's original vToken position.

For example :

- User has a 100e18 vUSDT position, with 40 % borrowing capacity used and collateral factor == 80 %;
- User wants to migrate all of it;
- User calls `supplyFromCollateral()`, which calculates potential shortfall against a redemption of the entire 100e18 vUSDT position;
- Shortfall returns true so USDT equivalent of entire 100e18 vUSDT is flashloaned, and flashloan fees is charged on full amount;
- Rather if only the shortfall amount is flashloaned, user would lose lesser amount of funds to the flashloan fees;

User can already avoid a part of the fees by splitting the migration into two : one that doesn't cause a shortfall in the above situation (migrating 50e18 vUSDT first), and the next migration charging flashloan fees on the other 50e18 vUSDT.

We can modify the logic to charge flashloan fees only on actual loan amount needed to cover the shortfall.

Recommendation: Since user is the one paying for the flashloan, the fee mechanism can be improved to favour the user's migration.

Venus Protocol: Acknowledged.

Cantina Managed: Acknowledged. A loan sized to the shortfall would be sufficient, but today the full-size loan carries no fee: `flashLoanFeeMantissa` is 0 on the Core USDT, USDC and U markets, so the premium is 0. Users can already limit the fee by splitting the migration. The full-amount loan will be kept for now and revisited if a flash-loan fee is set on these markets.